

Air University



Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: May 27, 2025

Experiment No. 12

Title: Introduction to Verilog II (Sequential Circuits: SR Latch and Clocked SR Flip-Flop)

Objectives

- To enhance comprehension of sequential circuits and their essential role in digital systems.
- To model and simulate SR Latch and Clocked SR Flip-Flop circuits at the gate level using Verilog.
- To examine and contrast the functionalities of asynchronous (SR Latch) and synchronous (Clocked SR Flip-Flop) circuits.
- To verify the operation of both circuits through simulation, focusing on the outputs Q and $\bar{Q}$ ($\sim Q$ or q_1).

Equipment and Components

- A computer with ModelSim or a compatible Verilog simulator
- A Verilog source code editor

Introduction

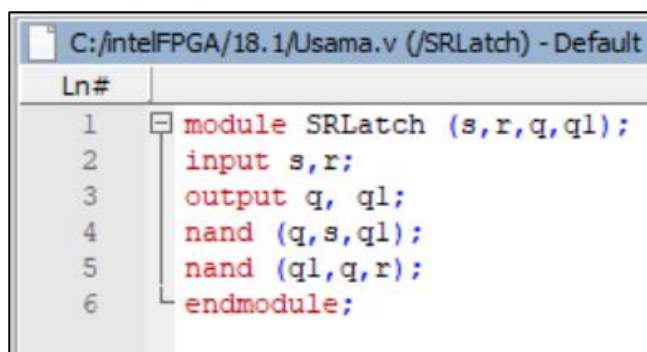
Sequential circuits are key elements in digital electronics that integrate logic operations with memory components, enabling the storage and processing of data over time. Unlike combinational circuits, which rely solely on current inputs, sequential circuits depend on both current inputs and previous states. This characteristic allows for the design of memory devices, registers, and control logic systems. The SR Latch operates as an asynchronous memory element that alters its state immediately based on input signals, while the Clocked SR Flip-Flop functions as a synchronous device, updating its output only in response to a clock signal. This experiment focuses on realizing both circuits in Verilog utilizing gate-level modeling, transitioning from hardware schematics to hardware description language (HDL) representations.

Procedure

A. SR Latch Implementation in Verilog

1. Module Design:

Begin by creating a Verilog module named `SRLatch`, which includes inputs `s` (Set) and `r` (Reset), as well as outputs `q` and `q1` (Q and $\bar{Q}$). Implement the cross-coupled NAND gate configuration as demonstrated in the following code:



```
C:/intelFPGA/18.1/Usama.v (/SRLatch) - Default
Ln#
1  module SRLatch (s,r,q,q1);
2      input s,r;
3      output q, q1;
4      nand (q,s,q1);
5      nand (q1,q,r);
6  endmodule;
```

2. Testbench Creation:

Build a testbench to apply all possible combinations of `s` and `r`, allowing for observation of both outputs `q` and `q1` for every input scenario.

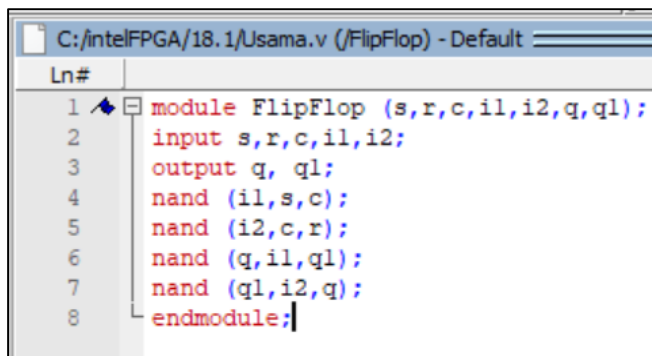
3. Simulation:

Conduct the simulation and take notes on the outputs for each input combination. Review the results to verify the behaviors associated with the set, reset, hold, and invalid states.

B. Clocked SR Flip-Flop Implementation in Verilog

1. Module Design:

Define a Verilog module named `FlipFlop`, which consists of inputs `s`, `r`, `c` (clock), `i1`, and `i2`, alongside outputs `q` and `q1`. Implement the configuration using four NAND gates, as illustrated in the following code:



```

C:/intelFPGA/18.1/Usama.v (FlipFlop) - Default
Ln#
1 module FlipFlop (s,r,c,i1,i2,q,q1);
2   input s,r,c,i1,i2;
3   output q, q1;
4   nand (i1,s,c);
5   nand (i2,c,r);
6   nand (q,i1,q1);
7   nand (q1,i2,q);
8   endmodule;
  
```

2. Testbench Creation:

Create a testbench to toggle the inputs `s`, `r`, and `c` (clock). Observe and document the outputs `q` and `q1` during all pertinent input and clock situations.

3. Simulation:

Carry out the simulation and confirm that the outputs change solely when the clock is active (synchronous operation), validating the proper functionality for each input combination.

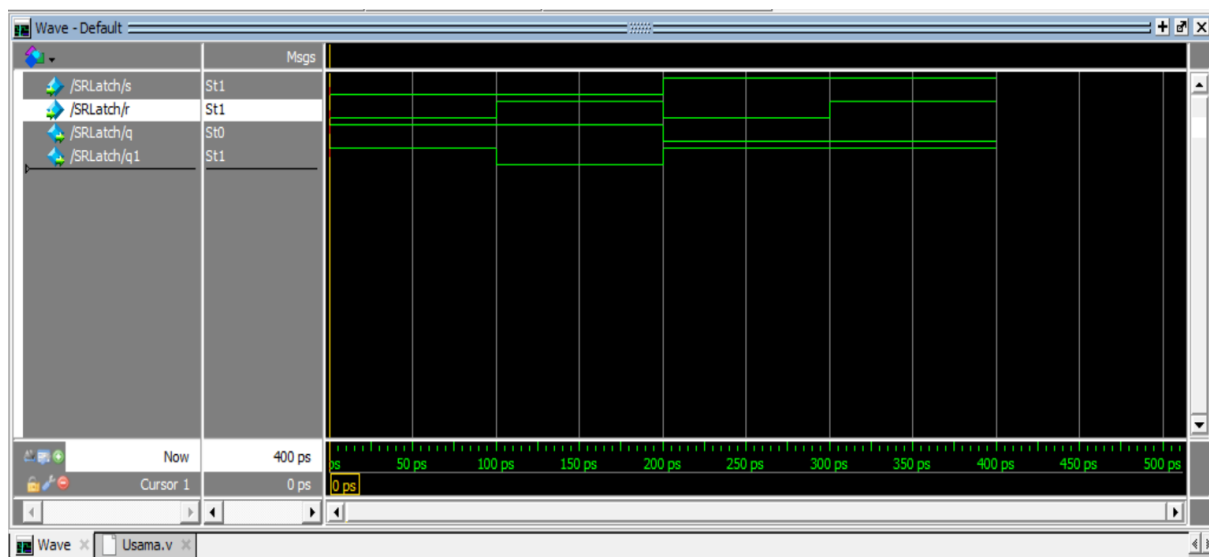
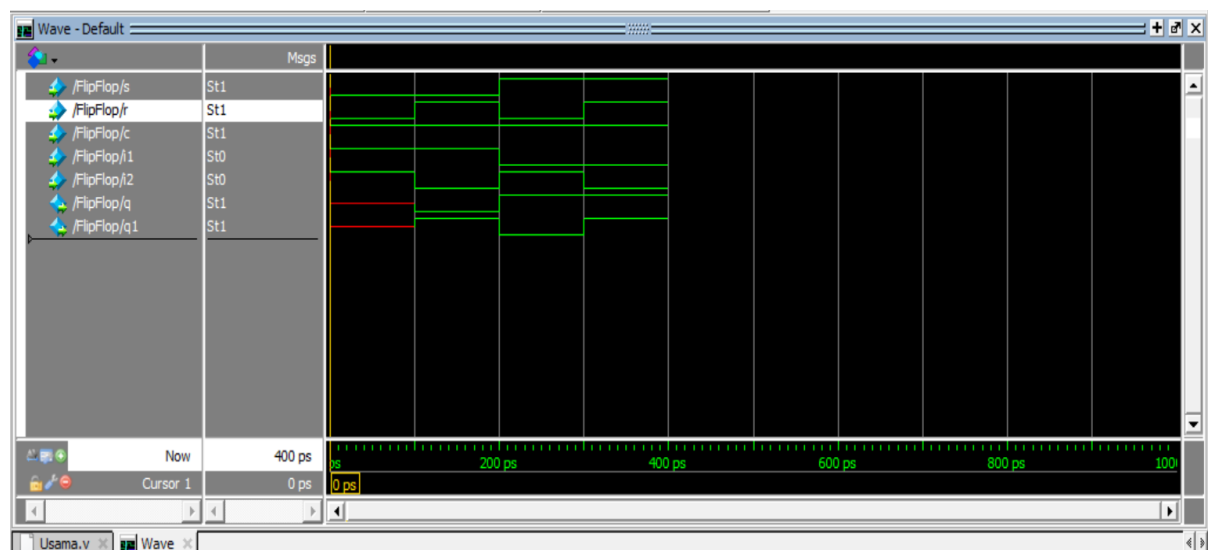
Truth Tables

A. SR Latch (NAND-based):

S	R	$Q_{(n+1)}$	Description
0	0	Invalid	Forbidden state
0	1	1	Set
1	0	0	Reset
1	1	$Q_{(n)}$	Hold / No Change

B. Clocked SR Flip-Flop (NAND-based):

Clock	S	R	$Q_{(n+1)}$	Description
1	0	0	$Q_{(n)}$	Hold
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	Invalid	Forbidden state

Simulated Results**A. SR Latch (NAND-based):****B. Clocked SR Flip-Flop (NAND-based):**

Conclusions

This experiment successfully demonstrated the modeling and simulation of sequential circuits in Verilog using gate-level primitives. The SR Latch highlighted the characteristics of a fundamental asynchronous memory element, while the Clocked SR Flip-Flop illustrated how a clock signal can synchronize state changes. The gate-level Verilog representations reflect the foundational hardware schematics, underscoring the connection between HDL code and physical digital circuits. Grasping these essential circuits is crucial for developing more advanced sequential logic in digital systems.

THE END
